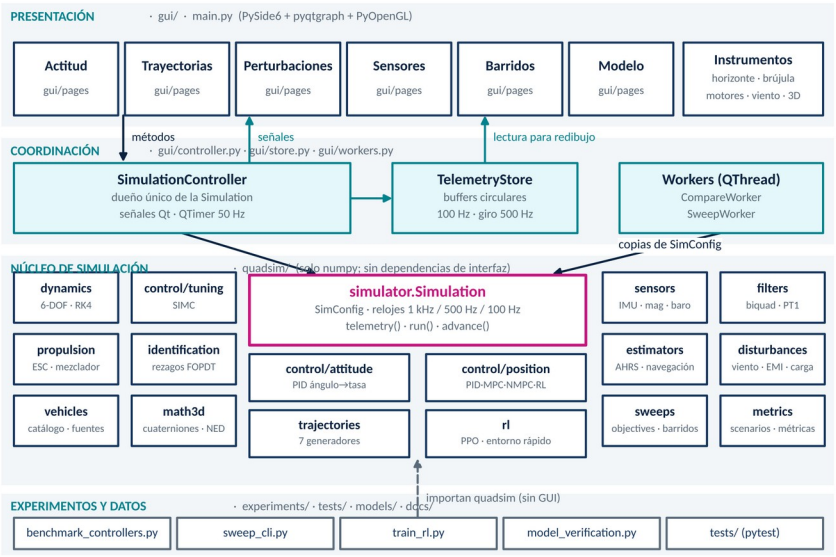
REGISTRO DE SOPORTE LÓGICO · DIRECCIÓN NACIONAL DE DERECHO DE AUTOR

QuadSIM-PRIMBIO

Simulador 6-DOF de cuadricóptero para sintonizar controladores en un entorno simulado, como punto de partida de la sintonía de drones reales

MANUAL TÉCNICO

Arquitectura, componentes, modelo matemático, algoritmos, temporización y despliegue del simulador.



Arquitectura en capas del software.

DOCUMENTO **Manual Técnico**	CÓDIGO **PQS-MT-002**	REVISIÓN **3.0**
SOFTWARE **QuadSIM-PRIMBIO v3.0**	FECHA **Octubre de 2026**	ESTADO **Emitido para registro**
AUTOR(ES) **Jesús Daniel Jácome (autor) · Francisco Javier Burgos Flórez (director)**	MODALIDAD **Proyecto Académico Especial (PAE)**	IDIOMA **Español**
AFILIACIÓN **Universidad Nacional de Colombia, Sede La Paz · Semillero PRIMBIO · Grupo de Investigación IAM-NANO**	REPOSITORIO **github.com/JesusJacome18/PAE-Quadcopter-SIM**	

Documento técnico de soporte para el registro de soporte lógico ante la Dirección Nacional de Derecho de Autor (Colombia). Las cifras de este documento provienen de ejecuciones del propio software y pueden reproducirse con los guiones del repositorio.

CONTROL DEL DOCUMENTO

Título	QuadSIM-PRIMBIO — Manual Técnico
Código / revisión	PQS-MT-002 · Rev. 3.0
Versión del software	3.0 (repositorio github.com/JesusJacome18/PAE-Quadcopter-SIM)
Elaboró	Jesús Daniel Jácome (autor) · Francisco Javier Burgos Flórez (director)
Revisó	Francisco Javier Burgos Flórez (director)
Contexto	Proyecto Académico Especial (PAE) de Ingeniería Mecatrónica, desarrollado en el Semillero de Investigación PRIMBIO, adscrito al Grupo de Investigación IAM-NANO
Clasificación	Uso académico. Documento de soporte para el registro de soporte lógico ante la DNDA.

HISTORIAL DE REVISIONES

Rev.	Fecha	Descripción del cambio	Elaboró
3.0	2026-10	Emisión inicial para el registro ante la DNDA. La numeración de la revisión coincide con la versión 3.0 del software.	Jesús Daniel Jácome

DOCUMENTOS DEL REGISTRO

Código	Documento	
PQS-DS-001	Descripción del Software	
PQS-MT-002	Manual Técnico	este documento
PQS-MU-003	Manual de Usuario	
PQS-TF-004	Título y Descripción de Funciones	

Contenido

1 Introducción....4
1.1 Propósito del software....4
1.2 Contexto general....4
1.3 Alcance del documento....4
1.4 Convenciones....4
2 Arquitectura del sistema....5
2.1 Descripción de la arquitectura....5
2.2 Componentes funcionales....6
2.3 Estructura del proyecto....7
2.4 Flujo general de datos....7
2.5 Hilos y procesos....8
3 Componentes del software....8
3.1 Núcleo de simulación (quadsim)....8
3.2 Interfaz gráfica (gui)....10
4 Modelo matemático....11
4.1 Estado....11
4.2 Cinemática....11
4.3 Dinámica de traslación y rotación....11
4.4 Propulsor y asignación de control....11
4.5 Aerodinámica, viento y suelo....12
4.6 Sensores e interferencia....12
4.7 Parámetros del vehículo de referencia....12
5 Algoritmos y métodos implementados....13
5.1 Integración numérica: Runge–Kutta de cuarto orden....13
5.2 Filtros digitales....13
5.3 Estimadores de actitud y de navegación....13
5.4 PID discreto y sintonía SIMC....14
5.5 Planitud diferencial....14
5.6 PID de posición en cascada....14
5.7 MPC lineal con previsión....14
5.8 MPC no lineal (iLQR)....15
5.9 Aprendizaje por refuerzo fuera de línea....15
5.10 Barridos de parámetros....15
6 Restricciones y consideraciones de tiempo real....16
6.1 Temporización....16
6.2 Ejecución concurrente....16
6.3 Desempeño medido....16
6.4 Limitaciones del modelo....17
6.5 Limitaciones de los controladores....17
6.6 Datos pendientes del dron real....17
7 Despliegue y ejecución....17
7.1 Entorno de desarrollo....17
7.2 Dependencias....18
7.3 Ejecución desde Python....18
7.4 Pruebas y verificación....18
7.5 Organización de datos y modelos....19
7.6 Ejecutable para Windows (PyInstaller)....19
7.7 Regeneración de estos documentos....19
7.8 Tecnologías utilizadas....20
8 Glosario de términos....20

1 Introducción

1.1 Propósito del software

QuadSIM-PRIMBIO simula un cuadricóptero con el nivel de detalle que necesita quien va a sintonizar su control: dinámica no lineal de seis grados de libertad, motores con su rezago, sensores con ruido y vibración, filtros, estimadores y el control en cascada de un autopiloto. Sobre esa base se comparan controladores de trayectoria, se aplican perturbaciones y se ejecutan barridos automáticos de parámetros. El objetivo de ingeniería es obtener en simulación una sintonía que sirva de punto de partida para el dron real.

1.2 Contexto general

El software es un Proyecto Académico Especial (PAE) de Ingeniería Mecatrónica, desarrollado en el Semillero de Investigación PRIMBIO, adscrito al Grupo de Investigación IAM-NANO, en la Universidad Nacional de Colombia, Sede La Paz (Valledupar, Cesar). La versión 3.0 separó la simulación de la telemetría del dron físico, que vive en otro proyecto, y reemplazó el modelo linealizado por eje de la versión anterior por un modelo Newton–Euler completo con traslación 3D, integrado con RK4. El vehículo de referencia, PAE-X450, toma todos sus parámetros de una misma fuente verificable: el modelo por defecto de CopterSim/RflySim (Quan y colaboradores, Universidad de Beihang).

1.3 Alcance del documento

Este manual describe cómo está construido el programa: su arquitectura, cada componente, el modelo matemático que programa, los algoritmos, las tasas de ejecución y el procedimiento de despliegue. Está dirigido a quien vaya a mantener o extender el código. La derivación completa del modelo y la evidencia numérica de cada bloque están en el documento del modelo matemático (3.ª edición); el uso de la interfaz se explica en el Manual de Usuario (PQS-MU-003).

1.4 Convenciones

Tabla 1 | Convenciones usadas en el código y en este documento.

Elemento	Convención
Marco inercial	NED: x norte, y este, z abajo. La altura es h = −z.
Marco cuerpo	FRD: x adelante, y derecha, z abajo (igual que PX4 y Quan).
Actitud	Cuaternión unitario (w, x, y, z); Euler Z-Y-X (roll φ, pitch θ, yaw ψ). El yaw es el rumbo, positivo en sentido horario visto desde arriba.
Motores	Quad X de PX4: M1 delante-derecha (CCW), M2 atrás-izquierda (CCW), M3 delante-izquierda (CW), M4 atrás-derecha (CW).
Mando	Acelerador $\sigma \in [0, 1]$ por motor; momento normalizado $u \in [-1, 1]$ en el lazo de tasa (convención PX4).
Unidades	SI en el núcleo (m, s, rad, N, N·m). La interfaz muestra grados y rpm donde es más legible.

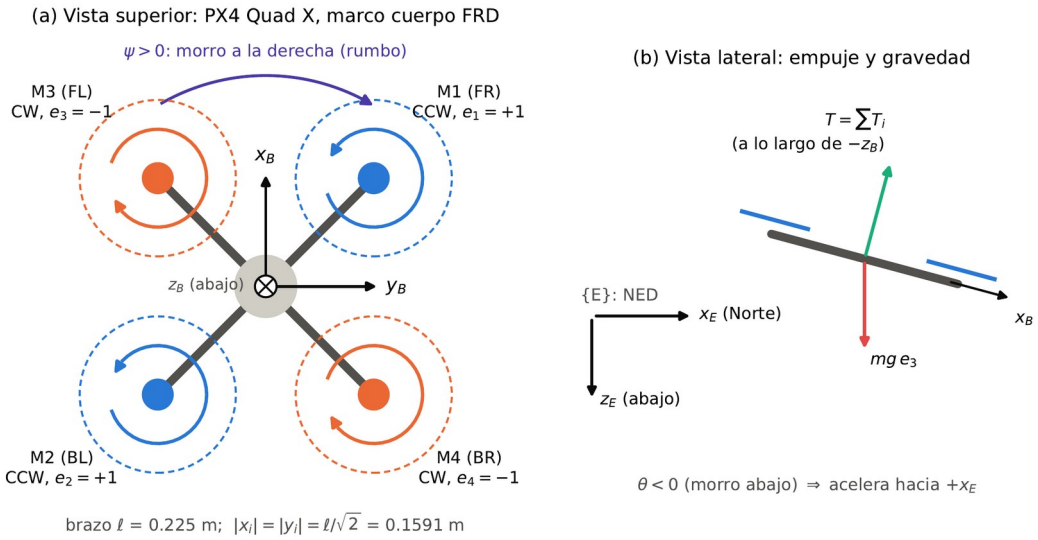


Figura 1 | Numeración y sentido de giro de los motores (a) y marcos de referencia (b). Figura del documento del modelo.

2 Arquitectura del sistema

2.1 Descripción de la arquitectura

El programa tiene cuatro capas (figura 2). La capa de presentación (`gui/`) contiene la ventana, las seis páginas y los instrumentos. La capa de coordinación tiene un único objeto dueño de la simulación, `SimulationController`, los buffers de telemetría y los hilos de trabajo. El núcleo (`quadsim/`) no importa nada de Qt y se puede usar desde cualquier guion. La última capa agrupa los guiones de experimentos, las pruebas, los modelos entrenados y la documentación.

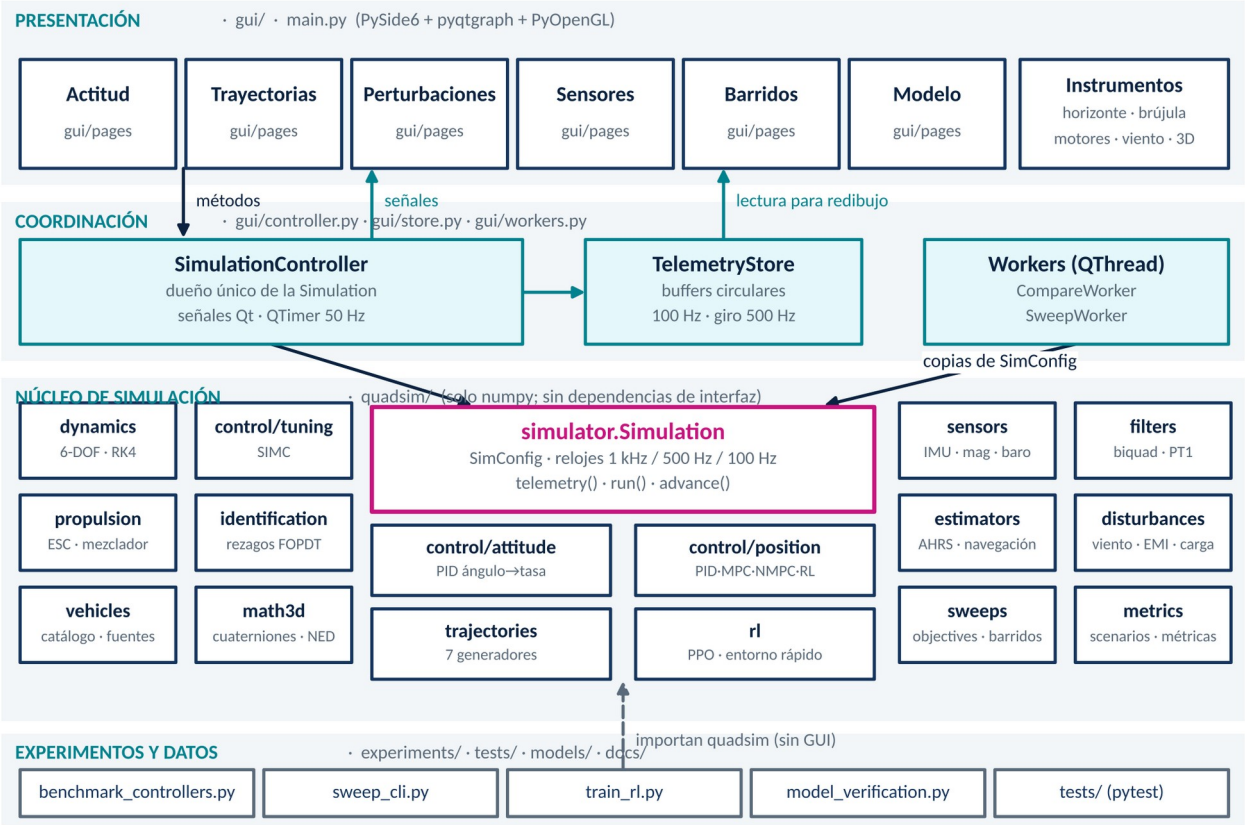


Figura 2 | Arquitectura en capas. Las flechas indican quién llama a quién.

La regla que sostiene esta división es que ninguna página toca el simulador. Las páginas piden cambios con métodos del controlador (`set_gain`, `set_filter`, `set_wind`, `fly_trajectory`…) y se enteran de lo que pasa por señales Qt (`telemetry`, `reset_done`, `vehicle_changed`, `estimator_changed`, `filters_changed`, `mode_changed`, `step_test_done`, `event`). Así se puede cambiar el vehículo, el estimador o el controlador a mitad de una corrida sin que un panel quede apuntando a un objeto que el reinicio ya reemplazó.

2.2 Componentes funcionales

Tabla 2 | Componentes funcionales y su ubicación.

Componente	Ubicación	Responsabilidad
Orquestador	quadsim/simulator.py	Une planta, sensores, estimación, control y perturbaciones con tres relojes.
Planta	dynamics.py, propulsion.py, vehicles.py	Modelo 6-DOF, propulsor, mezclador y catálogo de vehículos.
Medición	sensors.py, filters.py	Sensores con ruido, sesgo, retardo y vibración; filtros digitales.
Estimación	estimators.py	AHRS (Madgwick, Mahony, EKF) y filtro de navegación.
Control	control/	PID, SIMC, cascada de actitud, lazo vertical, planitud diferencial y controladores de posición.
Referencias	trajectories.py	Generadores de trayectoria con derivadas analíticas.
Entorno	disturbances.py	Viento, interferencia magnética, carga útil, fallas de motor.
Experimentación	scenarios.py, metrics.py, objectives.py, sweeps.py, identification.py, rl/	Inicialización común, métricas, barridos, identificación y aprendizaje.
Interfaz	gui/	Ventana, páginas, instrumentos, hilos y buffers.

2.3 Estructura del proyecto

```
PAE-Quadcopter-SIM/
├── main.py                  punto de entrada de la interfaz
├── requirements.txt         dependencias
├── quadsim/                 núcleo de simulación (solo NumPy)
│   ├── math3d.py            cuaterniones, Euler Z-Y-X, rotaciones (NED/FRD)
│   ├── vehicles.py          catálogo de vehículos con fuentes
│   ├── propulsion.py        ESC + motor, mezclador con desaturación
│   ├── dynamics.py          modelo 6-DOF, RK4 y Euler
│   ├── sensors.py           IMU, magnetómetro, barómetro, posición
│   ├── filters.py           Biquad y PT1
│   ├── estimators.py        Madgwick, Mahony, EKF, navegación
│   ├── disturbances.py      viento, EMI, carga útil, falla de motor
│   ├── simulator.py         SimConfig, Simulation (tres relojes)
│   ├── trajectories.py      7 generadores de referencia
│   ├── control/             pid, tuning (SIMC), attitude, position/
│   ├── identification.py    rezagos del lazo interno (FOPDT)
│   ├── scenarios.py         inicialización común y escenarios
│   ├── metrics.py           métricas de escalón y seguimiento
│   ├── objectives.py        escenarios y métricas de barrido
│   ├── sweeps.py            motor de barridos de parámetros
│   └── rl/                  entorno rápido, PPO, políticas, validación
├── gui/                     interfaz PySide6
│   ├── main_window.py       ventana, barra superior, riel, registro
│   ├── controller.py        SimulationController
│   ├── store.py · workers.py · registry.py · analysis.py
│   ├── theme.py · icons.py
│   ├── pages/               seis áreas de trabajo
│   └── widgets/             instrumentos, gráficas, vista 3D, formularios
├── experiments/             banco, barridos, RL, verificación del modelo
│   └── results/             resultados generados por los guiones
├── models/                  pesos de las políticas de RL (.npz)
├── tests/                   pruebas automáticas (pytest)
└── docs/                    modelo (PDF/LaTeX), guías, capturas, registro
```

2.4 Flujo general de datos

Un paso de control (2 ms de tiempo simulado) recorre siempre la misma secuencia, dentro de `Simulation.step_control()`:

1. Las perturbaciones se actualizan (viento, eventos de carga y de falla programados).
2. Los sensores muestrean el estado verdadero: IMU con vibración, magnetómetro (con interferencia si está activa), barómetro y posición externa, cada uno con su tasa y su línea de retardo.
3. El giroscopio pasa por los dos biquads de cada eje; el acelerómetro, por un pasa-bajos de 30 Hz antes del AHRS.
4. El estimador de actitud se actualiza, con la compensación de aceleración cinemática solo si hay correcciones de posición absoluta; el filtro de navegación predice con la aceleración medida y corrige con la posición y el barómetro.
5. Cada cinco pasos de control (100 Hz) corre el lazo externo: el lazo vertical en modo actitud, o el controlador de posición en modo trayectoria, que entrega una aceleración deseada convertida en empuje y actitud por planitud diferencial.
6. La cascada ángulo → tasa produce el momento normalizado; el mezclador lo convierte, junto con el empuje, en el acelerador de cada motor.
7. La planta avanza dos subpasos de RK4 a 1 kHz con el acelerador retenido.

La interfaz lee el estado con `Simulation.telemetry()`, un diccionario plano con verdad, estimación, referencias, mandos, perturbaciones y métricas del mezclador. El controlador lo guarda a 100 Hz de tiempo

simulado en `TelemetryStore` y guarda aparte el giroscopio crudo y filtrado a 500 Hz, porque a 100 Hz el tono de 177 Hz de las hélices sería invisible por Nyquist.

2.5 Hilos y procesos

El hilo principal de Qt corre la simulación en vivo con un `QTimer` de 50 Hz. En cada tic se avanza Δt_sim = Δt_real × velocidad en pasos de 2 ms. Si el cómputo excede el presupuesto de 28 ms, el tic se corta y no se acumula deuda: baja el factor de tiempo real y la ventana sigue respondiendo. Solo la página visible se redibuja, a lo sumo a 30 Hz. Las comparaciones de controladores y los barridos corren en `QThread` sobre copias profundas de la configuración; los barridos pueden repartir corridas en procesos con `multiprocessing` y arranque `spawn` (figura 3).

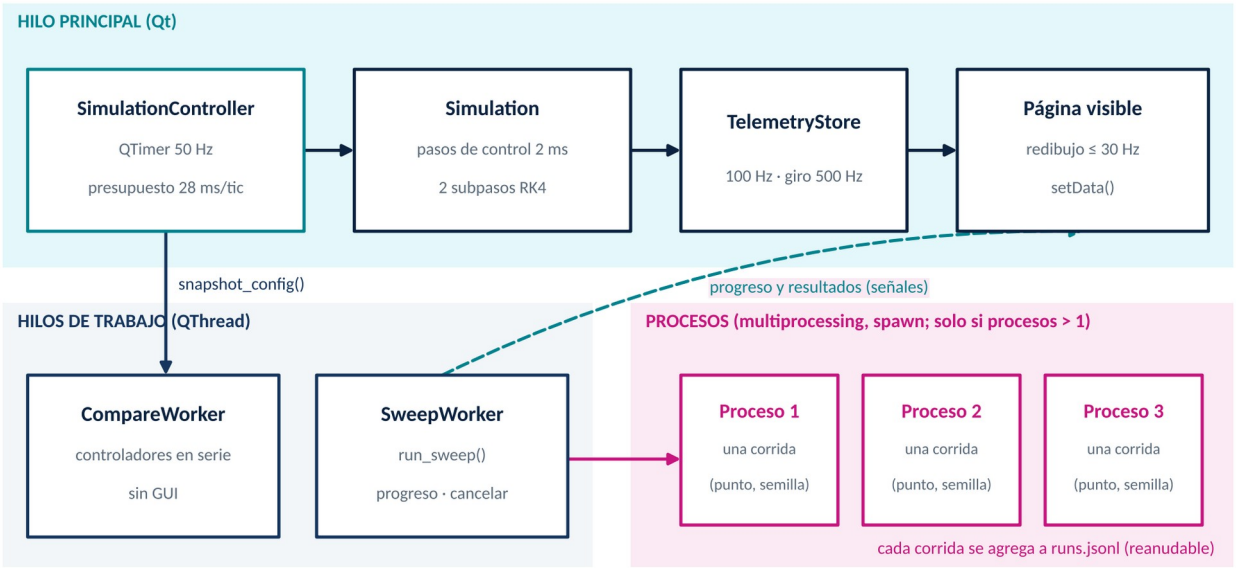


Figura 3 | Hilo principal, hilos de trabajo y procesos de los barridos.

3 Componentes del software

3.1 Núcleo de simulación (quadsim)

Tabla 3 | Módulos del núcleo y sus elementos principales.

Módulo	Elementos principales	Función
math3d	q_mul, q_to_rot, euler_to_q, q_to_euler, q_derivative	Álgebra de cuaterniones y rotaciones Z-Y-X en NED/FRD.
vehicles	VehicleParams, VEHICLES, get_vehicle, with_payload	Catálogo de 5 vehículos, cada uno con fuente y supuestos; cantidades derivadas (hover, empuje máximo, momentos máximos).
propulsion	esc_steady_speed, esc_inverse, Mixer	Recta del ESC con zona muerta; asignación con desaturación por prioridades.
dynamics	QuadrotorDynamics, rk4_step, euler_step, Environment	Derivada del estado de 17 componentes, efecto suelo, contacto con el suelo.
sensors	SensorConfig, SensorSuite, ImuSample	IMU, magnetómetro, barómetro y posición con tasas, ruido, sesgo, retardos y vibración de paso de pala.
filters	Biquad, PT1	Filtros de segundo orden (forma directa II traspuesta) y de primer orden; respuesta y retardo de grupo.
estimators	MadgwickEstimator, MahonyEstimator, EKFEstimator, IdealEstimator, NavEstimator	AHRS intercambiables y filtro de Kalman de posición y velocidad.

Módulo	Elementos principales	Función
disturbances	WindConfig, MagEMIConfig, PayloadEvent, MotorFault, DisturbanceManager	Viento medio, Dryden, ráfaga, EMI, carga útil y fallas.
control.pid	PID, PIDGains	PID discreto con derivada sobre la medida filtrada y anti-windup.
control.tuning	suggest_attitude_gains, suggest_vertical_gains	Ganancias SIMC calculadas del modelo.
control.attitude	AttitudeController, VerticalController, acceleration_to_attitude	Cascada de actitud, lazo vertical y planitud diferencial.
control.position	PIDPositionController, LinearMPC, NonlinearMPC, RLPositionController, MPCRLController	Controladores de posición registrados en CONTROLLERS.
trajectories	Hover, StepWaypoints, Circle, Lemniscate, Helix, MinimumSnap, Quintic	Referencias con derivadas analíticas, registradas en TRAJECTORIES.
simulator	SimConfig, Simulation, NavState, PositionCommand	Orquestación, telemetría y registro.
scenarios	make_config, prepare_tracking_sim, make_tracking_sim	Inicialización idéntica para GUI, banco, barridos y pruebas.
metrics	step_metrics, tracking_metrics	Métricas sobre la posición verdadera.
identification	identify, fit_fopdt, get_lags	Rezagos del lazo interno vistos desde el lazo de posición.
objectives	AttitudeStep, HoverDisturbance, EstimatorAccuracy	Escenarios cortos y deterministas para barridos.
sweeps	SweepParam, SweepSpec, run_sweep, SweepResult, PRESETS	Diseño de experimentos, ejecución paralela reanudable y análisis.
rl	env, ppo, policy, surrogate, base_mpc, refgen, validation	Aprendizaje por refuerzo fuera de línea.

3.1.1 Orquestador: simulator.Simulation

`SimConfig` reúne en un solo objeto todo lo que define una corrida: vehículo, tasas (`physics_hz` = 1000, `ctrl_hz` = 500, `outer_hz` = 100), integrador, estimador y sus ganancias, sensores, perturbaciones, filtros del giroscopio, modo (`attitude` o `trajectory`), arranque en el aire y semilla. `Simulation` construye a partir de ella todos los objetos y expone `reset()`, `step_control()`, `advance(t)`, `run(t)`, `telemetry()` y `log_arrays()`. Los métodos `set_estimator`, `set_filter`, `set_vehicle`, `set_position_controller`, `set_trajectory` y `attach_payload` permiten cambiar la configuración en caliente. El arranque en el aire llena las líneas de retardo de los sensores con el estado inicial, alinea el estimador con la actitud verdadera y lleva la covarianza de navegación a régimen, como haría la alineación en tierra de un autopiloto.

3.1.2 Controladores de posición

Todos heredan de `PositionController` y reciben un `NavState` con lo que un controlador real puede ver: posición, velocidad, actitud y aceleración *estimadas* y la masa nominal. Devuelven un `PositionCommand` con la aceleración deseada en NED, el rumbo y su prealimentación. Comparten la misma restricción de inclinación máxima (30°) y la misma caja de aceleración, de modo que la comparación entre ellos es justa.

Tabla 4 | Controladores de posición registrados.

Clave	Etiqueta en la interfaz	Parámetros	Archivo
`pid`	PID en cascada (PX4)	15	control/position/pid.py
`mpc`	MPC lineal con previsión	17	control/position/mpc.py
`nmpc`	MPC no lineal (iLQR)	19	control/position/nmpc.py
`rl`	RL puro (PPO offline)	2	control/position/rl.py
`mpc_rl`	Híbrido MPC + RL residual	2	control/position/mpc_rl.py

3.1.3 Trayectorias

Cada generador define `sample(t)` y devuelve un `RefPoint` con posición, velocidad, aceleración, jerk y rumbo, todos con derivadas analíticas. Las curvas periódicas arrancan desde el reposo con una rampa suave de la fase, de modo que la referencia es continua en aceleración desde el primer instante.

Tabla 5 | Generadores de trayectoria y sus parámetros.

Clave	Etiqueta	Parámetros (valor por defecto)
`hover`	Vuelo estacionario	Posición x (norte) 0 m; Posición y (este) 0 m; Altura 1.5 m; Rumbo 0 °; Duración 20 s
`step_waypoints`	Escalones (caja)	Lado de la caja 1 m; Escalón de altura 0.5 m; Altura inicial 1.5 m; Tiempo por consigna 4 s; Inicio x 0 m; Inicio y 0 m; Rumbo 0 °
`circle`	Círculo	Radio 1.5 m; Periodo de una vuelta 10 s; Número de vueltas 2; Rampa de arranque 3 s; Altura 1.5 m; Centro x (norte) 0 m; Centro y (este) 0 m; Rumbo 0 °
`lemniscate`	Figura en 8 (lemniscata)	Largo de cada lóbulo 2 m; Ancho de los lóbulos 1 m; Periodo de una vuelta 10 s; Número de vueltas 2; Rampa de arranque 3 s; Altura 1.5 m; Centro x (norte) 0 m; Centro y (este) 0 m; Rumbo 0 °
`helix`	Hélice	Radio 1.5 m; Subida por vuelta 0.5 m; Periodo de una vuelta 10 s; Número de vueltas 2; Rampa de arranque 3 s; Altura 1.5 m; Centro x (norte) 0 m; Centro y (este) 0 m; Rumbo 0 °
`minimum_snap`	Mínimo snap (waypoints)	Escala del recorrido 1 m; Velocidad media 0.8 m/s; Tiempo mínimo por tramo 1.5 s; Altura base 1.5 m; Inicio x 0 m; Inicio y 0 m; Rumbo 0 °
`quintic`	Punto a punto (mínimo jerk)	Inicio x 0 m; Inicio y 0 m; Altura inicial 1.5 m; Destino x 3 m; Destino y 2 m; Altura final 2 m; Tiempo de maniobra 4 s; Espera antes/después 2 s; Rumbo 0 °

3.2 Interfaz gráfica (gui)

Tabla 6 | Módulos de la interfaz.

Archivo	Contenido
main.py	Punto de entrada: opciones `--light`, `--vehicle`, `--play`; escalado de alta densidad; crea `MainWindow`.
gui/main_window.py	Ventana principal: barra superior, riel de navegación, pila de páginas, registro de eventos y atajos.
gui/controller.py	`SimulationController`: único dueño de la `Simulation`, reloj de la GUI, pruebas de escalón, perturbaciones, trayectorias y exportación.
gui/store.py	Buffers circulares de NumPy de tamaño fijo (600 s a 100 Hz) y anillo del giroscopio a 500 Hz; exportación CSV.
gui/workers.py	`CompareWorker` y `SweepWorker` en `QThread`.
gui/registry.py	Acceso tolerante a los registros del núcleo: si un módulo no importa, la página lo informa en lugar de caerse.
gui/analysis.py	Cálculos para las gráficas (espectro, Bode, métricas).
gui/theme.py, gui/icons.py	Tokens de color y tipografía, hojas de estilo oscura y clara; iconos vectoriales.
gui/pages/	Actitud, Trayectorias, Perturbaciones, Sensores, Barridos y Modelo.
gui/widgets/	Horizonte artificial, brújula, barras de motores, dial de viento (QPainter); lienzos pyqtgraph; vista 3D OpenGL con respaldo isométrico; formularios generados desde `PARAMS`.
gui/tour.py	Tour guiado (37 pasos declarados como datos `TourStep`): capa que oscurece la ventana y resalta cada elemento, con tarjeta y navegación por teclado.
gui/scenario.py	Escenarios en JSON: validación, aplicación a la sesión (`apply`), captura de la sesión (`capture`), acciones programadas en tiempo simulado y lista de ejemplos.
gui/dialogs.py	Bienvenida, galería de ejemplos, tarjeta guía, guardar y abrir escenario, atajos y «Acerca de».
gui/about.py	Nombre, versión, autores y filiación del programa en un solo lugar.
gui/settings.py	Preferencias persistentes (por ejemplo, no volver a mostrar la bienvenida).

4 Modelo matemático

Esta sección resume las ecuaciones que resuelve el núcleo. Cada una corresponde a una función identificable del código; la derivación, las fuentes y la verificación están en el documento del modelo.

4.1 Estado

El estado tiene 17 componentes: posición y velocidad en NED, cuaternión de actitud, velocidad angular en el cuerpo y velocidad de los cuatro rotores.

$$x=\left[p^T\, v^T\, q^T\, \omega^T\, \Omega_1\, \Omega_2\, \Omega_3\, \Omega_4\right]^T \in R^{17} \tag{1}$$

4.2 Cinemática

$$\dot{p}=v\,,\ \dot{q}=1/2\, q\otimes\begin{bmatrix}0\\ \omega\end{bmatrix} \tag{2}$$

El cuaternión se renormaliza después de cada paso de integración. La matriz de rotación cuerpo → inercial, **R**(**q**), sigue la convención Z-Y-X.

4.3 Dinámica de traslación y rotación

$$m\,\dot{v}=m\,g\,e_3+R(q)\,f_B+F_{\text{ext}}\,,\ f_B=\begin{bmatrix}f_{a,x}\\ f_{a,y}\\ f_{a,z}-\sum_{i=1}^{4}T_i\end{bmatrix} \tag{3}$$

$$J\,\dot{\omega}=\tau_B-\omega\times(J\,\omega)-\omega\times h_r+\tau_a+\tau_{\text{ext}}\,,\ h_{r,z}=-J_m\sum_{i=1}^{4}e_i\,\Omega_i \tag{4}$$

$\mathbf{f}_a$ es el arrastre cuadrático del fuselaje calculado con la velocidad relativa al aire, $\mathbf{h}_r$ el momento angular de los rotores (efecto giroscópico), e_i = ±1 el sentido de giro de cada rotor y $\boldsymbol{\tau}_a$ el amortiguamiento rotacional. El término de reacción por aceleración de los rotores se omite, porque el modelo de motor de primer orden no limita la corriente y daría una aceleración inicial irreal.

4.4 Propulsor y asignación de control

$$T_m\,\dot{\Omega}_i=\eta_i\,\omega_{\text{ss}}(\sigma_i)-\Omega_i\,,\ \omega_{\text{ss}}(\sigma)=max(0\,,C_R\,\sigma+\omega_b)\,,\ T_i=C_T\,\Omega_i^2\,,\ Q_i=C_M\,\Omega_i^2 \tag{5}$$

η_i es la eficiencia del motor (1 nominal; menor que 1 con una falla). El empuje total y los momentos se relacionan con los cuadrados de las velocidades por la matriz de asignación del bastidor en X:

$$\begin{bmatrix}T\\ \tau_x\\ \tau_y\\ \tau_z\end{bmatrix}=\begin{bmatrix}C_T & C_T & C_T & C_T\\ -y_1C_T & -y_2C_T & -y_3C_T & -y_4C_T\\ x_1C_T & x_2C_T & x_3C_T & x_4C_T\\ e_1C_M & e_2C_M & e_3C_M & e_4C_M\end{bmatrix}\begin{bmatrix}\Omega_1^2\\ \Omega_2^2\\ \Omega_3^2\\ \Omega_4^2\end{bmatrix} \tag{6}$$

El mezclador invierte esta relación y, si algún motor se sale de [0, 1], desatura con la prioridad de PX4: conserva roll y pitch, después el empuje y por último la guiñada. El factor `k_rp` informa cuánto se recortó el momento de roll y pitch.

4.5 Aerodinámica, viento y suelo

$$k_{ge}(h)=\frac{T_{IGE}}{T_{OGE}}=\frac{1}{1-\left(\frac{R_p}{4h}\right)^2},\ k_{ge}\le 1.4 \tag{7}$$

El efecto suelo de Cheeseman y Bennett es opcional. El viento entra a la dinámica como velocidad del aire: el arrastre se calcula con la velocidad relativa $\mathbf{v}-\mathbf{v}_w$, donde $\mathbf{v}_w$ suma el viento medio, la turbulencia de Dryden y una ráfaga 1−cos.

$$\dot{u}=-\frac{V}{L}u+\sigma\sqrt{\frac{2V}{L}}n(t),\ v_{raf}(t)=\frac{A}{2}\left(1-\cos\frac{2\pi(t-t_0)}{d}\right) \tag{8}$$

4.6 Sensores e interferencia

El ruido blanco de cada sensor se escala con su densidad espectral y la tasa de muestreo ($\sigma = N\cdot\sqrt{f_s}$); el sesgo del giroscopio arranca con un valor aleatorio y deriva como un paseo aleatorio. La vibración de las hélices agrega al giroscopio y al acelerómetro un tono a la frecuencia de paso de pala, que para hélices de dos palas es:

$$f_{bp}=\frac{2\Omega}{2\pi}\ \Omega=\Omega_h=557.3\,\text{rad/s}\Rightarrow f_{bp}\approx 177\,\text{Hz} \tag{9}$$

La interferencia electromagnética en el magnetómetro es un campo parásito proporcional a la corriente de los motores más pulsos aleatorios:

$$b_{EMI}=k_I\left(\frac{1}{4}\sum_{i=1}^{4}\sigma_i^{1.5}\right)\hat{d}_{EMI}+\text{pulsos} \tag{10}$$

SUPUESTO El exponente 1.5 de la interferencia no se deriva del modelo de propulsor; es un supuesto declarado en el documento del modelo (sección 13).

4.7 Parámetros del vehículo de referencia

Tabla 7 | Parámetros del PAE-X450 (fuente: CopterSim/RflySim, Quan et al.).

Parámetro	Símbolo	Valor	Unidad
Masa	m	1.4	kg
Brazo (centro a motor)	ℓ	0.225	m
Inercias	J_{xx}, J_{yy}, J_{zz}	0.0241, 0.0239, 0.0386	kg·m²
Coeficiente de empuje	C_T	1.105×10^{-5}	N/(rad/s)²
Coeficiente de par	C_M	1.779×10^{-7}	N·m/(rad/s)²
Inercia del rotor	J_m	1.287×10^{-4}	kg·m²
Recta del ESC	C_R, ω_b	1148, -141.4	rad/s
Constante de tiempo del motor	T_m	0.02	s
Velocidad de hover	Ω_h	557.3 (5322 rpm)	rad/s
Acelerador de hover	σ_h	0.609	—
Relación empuje/peso	—	3.26	—
Momentos máximos	τ_{max}	3.5627 / 3.5627 / 0.3605	N·m

5 Algoritmos y métodos implementados

5.1 Integración numérica: Runge–Kutta de cuarto orden

$$x_{n+1} = x_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right),\ k_1 = f\left(x_n\right), k_{2,3} = f\left(x_n + h/2\, k_{1,2}\right), k_4 = f\left(x_n + h\, k_3\right) \tag{11}$$

Con h = 1 ms y el mando retenido entre muestras de control. La verificación midió un orden de convergencia de 4.19 y una deriva relativa de energía en rotación libre del orden de 10^{-14}; el integrador de Euler (`euler_step`) queda disponible para comparación (figura 4).

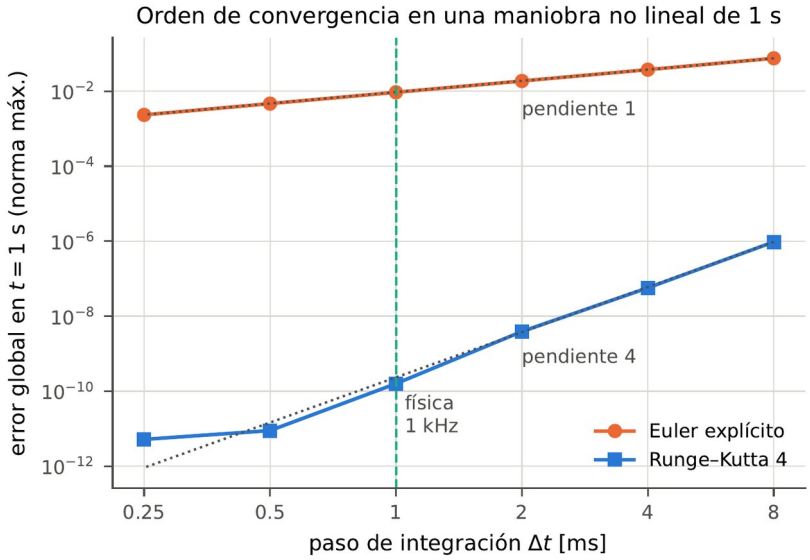


Figura 4 | Error contra paso de integración para Euler y RK4 (documento del modelo).

5.2 Filtros digitales

Cada biquad se evalúa en forma directa II traspuesta, con coeficientes de las fórmulas de Bristow-Johnson (pasa-bajos con Q, Butterworth con Q = 0.7071, Chebyshev tipo I con rizado y notch). Al rediseñar un filtro en vuelo se conservan los estados internos para no introducir un transitorio.

$$y_k = b_0 x_k + z_1,\ z_1 \leftarrow b_1 x_k - a_1 y_k + z_2,\ z_2 \leftarrow b_2 x_k - a_2 y_k \tag{12}$$

5.3 Estimadores de actitud y de navegación

Tabla 8 | Estimadores disponibles y ganancias por defecto.

Estimador	Principio	Ganancias por defecto
Madgwick	Descenso de gradiente sobre el error de gravedad y campo magnético; corrección proporcional a β.	β = 0.015
Mahony	Filtro complementario no lineal en SO(3) con corrección PI; el integral estima el sesgo del giroscopio.	k_p = 1.0, k_i = 0.05
EKF (7 estados)	Cuaternión y sesgo del giroscopio; predicción con el giroscopio, corrección con acelerómetro y magnetómetro.	σ_{giro} = 0.02, deriva 2×10^{-4}, σ_{acel} = 0.5, σ_{mag} = 0.3
Ideal	Devuelve la actitud verdadera; sirve para medir el costo de estimar.	—
Navegación	Kalman lineal de posición y velocidad por eje: predice con la aceleración medida rotada a NED y corrige con el sensor de posición y el barómetro.	—

Todos los AHRS reciben el acelerómetro filtrado a 30 Hz y, si está activa, la compensación de aceleración cinemática (como `ATT_ACC_COMP` de PX4): se resta la aceleración estimada por navegación para recuperar la dirección de la gravedad, porque el acelerómetro de un multirrotor que acelera no la mide. La resta usa la actitud estimada. Como el filtro de navegación predice con ese mismo acelerómetro, entre correcciones de posición el AHRS recibiría su propia estimación de la gravedad; sin sensor de posición la inclinación dejaría de ser observable y derivaría. Por eso la compensación solo se aplica cuando el filtro de navegación recibe correcciones de posición absoluta.

$$e=\hat{s}_g\times\left(R^T e_3\right)+\hat{m}\times\left(R^T b\right),\ \omega_c=\omega_{\text{med}}+k_p e+I,\ I\leftarrow I+k_i e\,\Delta t \tag{13}$$

5.4 PID discreto y sintonía SIMC

El PID usa la derivada de la medida (no del error) filtrada con un pasa-bajos de primer orden, integración condicional como anti-windup y un límite absoluto del integrador. Las ganancias iniciales del lazo de tasa salen de las reglas SIMC aplicadas a un integrador con retardo, cuyo retardo equivalente suma la latencia de la IMU, la retención, el filtro y la mitad de la constante del motor:

$$\frac{\omega(s)}{u(s)}=\frac{K\,e^{-\theta s}}{s},\ K=\frac{\tau_{\max}}{J},\ K_c=\frac{1}{K\left(\tau_c+\theta\right)},\ T_i=4\left(\tau_c+\theta\right),\ \tau_c=\theta \tag{14}$$

Para el PAE-X450 resulta θ = 16 ms y, en roll, K_p = 0.2114, K_i = 1.6515, K_d = 0.00338 en el lazo de tasa y K_p = 7.812 1/s en el de ángulo. Un escalón de 10° sube en 0.13 s sin sobrepaso apreciable.

5.5 Planitud diferencial

Los controladores de posición entregan una aceleración deseada. La fuerza necesaria se limita para respetar la inclinación máxima y se convierte en empuje y actitud de referencia:

$$F=m\left(a_{\text{des}}-g\,e_3\right),\ z_B=-\frac{F}{\|F\|},\ T=\|F\| \tag{15}$$

5.6 PID de posición en cascada

$$v_{\text{sp}}=v_{\text{ref}}+sat\left(K_p\left(p_{\text{ref}}-\hat{p}\right)\right),\ a_{\text{des}}=sat\left(a_{\text{ref}}+K_v e_v+K_i\int e_v\,dt+K_d\dot{e}_v\right) \tag{16}$$

Es la estructura de `mc_pos_control` de PX4, con prealimentación de la velocidad y la aceleración de referencia. Las ganancias por defecto en XY (K_p = 3.6, K_v = 5.5, K_i = 2.0) se afinaron en simulación; el integral se dejó bajo porque con interferencia magnética un integral grande entra en espiral.

5.7 MPC lineal con previsión

Por eje, el modelo de predicción tiene como estados posición, velocidad y aceleración, con el rezago identificado del lazo interno; se discretiza de forma exacta con retención de orden cero. El costo sobre N = 20 pasos de 50 ms (1 s de previsión) es:

$$J=\sum_{k=1}^{N} q_p\left(p_k-p_{\text{ref},k}\right)^2+q_v\left(v_k-v_{\text{ref},k}\right)^2+\sum_{k=0}^{N-1} r_u\left(u_k-u_{\text{ref},k}\right)^2+r_{\text{du}}\left(u_k-u_{k-1}\right)^2 \tag{17}$$

Eliminando los estados, el problema es un QP en las N entradas con restricciones de caja. Se calcula primero el óptimo sin restricciones; si viola la caja, se resuelve con FISTA (Beck y Teboulle) con reinicio por gradiente, precondicionamiento de Jacobi, arranque en caliente y a lo sumo 60 iteraciones. Un observador de Luenberger de perturbación de entrada (polos en −2.4 ± 1.8j) hace el control libre de offset ante viento y carga.

5.8 MPC no lineal (iLQR)

Usa un modelo de nueve estados (posición, velocidad, roll, pitch y empuje) con proyección trigonométrica exacta del empuje y arrastre cuadrático. Los estados de rezago se integran exactamente y la traslación con RK4. El problema se resuelve con iLQR con entradas acotadas a la manera de box-DDP: jacobianos por diferencias finitas vectorizadas, QP de caja de dimensión 3 por Newton proyectado en el paso hacia atrás y búsqueda de línea en lote en el paso hacia adelante. Se hace una iteración por llamada con arranque en caliente (iteración en tiempo real).

5.9 Aprendizaje por refuerzo fuera de línea

1. **Identificación.** Se excita el simulador completo con escalones aleatorios de aceleración y se ajusta un modelo retardo + primer orden por eje (`rl/surrogate.py`).
2. **Entorno rápido.** Solo traslación, 128 copias en paralelo en NumPy, con aleatorización de masa, rezagos, arrastre, viento, ráfagas y ruido (`rl/env.py`).
3. **PPO.** Actor y crítico de 2 × 64 neuronas tanh, GAE y recorte 0.2, escritos en NumPy con retropropagación manual verificada contra diferencias finitas (`rl/ppo.py`).
4. **Ejecución.** La política se carga desde `models/*.npz` y corre sin reentrenar en el simulador completo (`control/position/rl.py`). El híbrido suma al MPC una corrección acotada a 2 m/s² (`control/position/mpc_rl.py`).

$$a_{\text{des}} = \mathrm{sat}\left(a_{\text{MPC}} + s\,R\,\pi(o)\right),\ R = 2\,\text{m/s}^2,\ s \in [0, 1.5) \tag{18}$$

5.10 Barridos de parámetros

`sweeps.py` genera los puntos con rejilla completa, muestreo aleatorio o hipercubo latino y los refina alrededor del mejor (rejilla local o búsqueda de Hooke y Jeeves). Cada punto se evalúa con varias semillas usando números aleatorios comunes. El costo se normaliza contra la configuración por defecto corrida con las mismas semillas, de modo que J = 1 es «igual que el defecto»:

$$J = \sum_k w_k \frac{m_k}{m_k^{\text{base}}},\ J = \infty \text{ si hay choque} \tag{19}$$

El análisis entrega el ranking, el frente de Pareto de dos métricas, los efectos principales y la sensibilidad por ρ de Spearman. Los barridos predefinidos son:

Tabla 9 | Barridos predefinidos (`quadsim.sweeps.PRESETS`).

Clave	Descripción
`gyro_lpf`	Pasa-bajos del giroscopio (Fc × tipo)
`notch`	Notch del giroscopio (centro × Q)
`madgwick_beta`	β de Madgwick
`mahony_gains`	Ganancias Kp × Ki de Mahony
`ekf_noise`	Ruidos de medida del EKF
`rate_pid`	PID de tasa (Kp × Kd)
`hover_lpf`	Pasa-bajos bajo turbulencia
`overnight`	Nocturno: 8 parámetros conjuntos (LHS)

6 Restricciones y consideraciones de tiempo real

6.1 Temporización

El simulador reproduce los relojes de un autopiloto (figura 5). La pantalla es solo un observador: dibujar a 50 Hz no cambia la física.

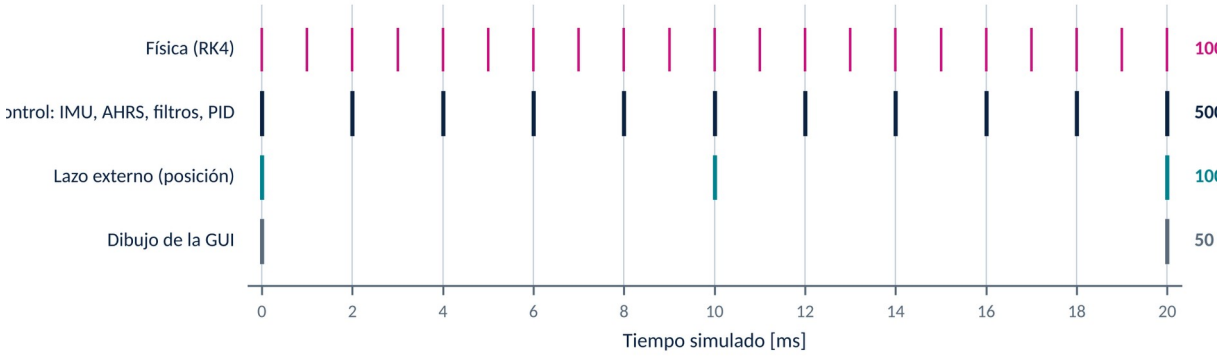


Figura 5 | Ejecuciones de cada lazo en una ventana de 20 ms de tiempo simulado.

Tabla 10 | Tasas de ejecución.

Lazo	Tasa	Periodo	Qué corre
Física	1000 Hz	1 ms	RK4 sobre el modelo 6-DOF (2 subpasos por paso de control)
Control	500 Hz	2 ms	Muestreo de IMU, filtros, AHRS, navegación, cascada de actitud, mezclador
Lazo externo	100 Hz	10 ms	Lazo vertical o controlador de posición
Telemetría	100 Hz	10 ms	Registro en buffers (500 Hz para el giroscopio)
Reloj de la GUI	50 Hz	20 ms	Avance de la simulación en vivo
Redibujo	≤ 30 Hz	≥ 33 ms	Solo la página visible

6.2 Ejecución concurrente

Las comparaciones corren los controladores en serie dentro de un `QThread` para no competir por CPU con la simulación en vivo. Los barridos corren en un `QThread` con un proceso por defecto; con más procesos usan un `Pool` de `multiprocessing` y fijan una hebra BLAS por proceso. Todas las corridas en segundo plano trabajan sobre copias de la configuración, nunca sobre la simulación en vivo.

6.3 Desempeño medido

Tabla 11 | Desempeño medido en la máquina de desarrollo.

Medida	Valor	Fuente
Factor de tiempo real sin interfaz (Mahony)	13.5×	model_verification.py, bloque L
Cómputo del MPC lineal por llamada	0.36 ms (media)	banco final
Cómputo del MPC no lineal por llamada	4.5 ms (media)	banco final
Memoria de la interfaz en uso	≈200 MB	medición con getrusage
Memoria del núcleo sin interfaz	≈32 MB	medición con getrusage

6.4 Limitaciones del modelo

- Sin dinámica eléctrica del motor ni modelo de batería: el voltaje es constante.
- Sin aerodinámica del rotor en vuelo de avance (*blade flapping*, arrastre inducido) ni anillo de vórtices.
- Empuje cuadrático puro; tensor de inercia diagonal, también con carga útil.
- Turbulencia con filtros de primer orden alineados con NED.
- Por defecto el mando se aplica en el mismo periodo en que se calcula y el ESC responde de forma continua, sin tasa de PWM ni cuantización. Las opciones `esc_rate_hz`, `compute_delay_samples` y `throttle_levels` de `SimConfig` permiten estudiarlos; quedan desactivadas hasta medir el ESC y el firmware del dron real, porque un periodo de cómputo resta unos 3° de margen de fase al lazo de tasa y un PWM de 50 Hz lo llevaría de 59° a unos 36°.
- La constante de tiempo del motor (20 ms) viene de la literatura y es igual al acelerar y al frenar; el lazo de tasa es sensible a ella.
- El lazo es perfectamente periódico (sin jitter del sistema operativo del ESP32) y el error de actitud se calcula en ángulos de Euler, válido con el límite de 30° de inclinación.
- El efecto suelo escala el empuje pero no el par de los rotores.
- Los rezagos identificados, los pesos de MPC y NMPC y las políticas de RL se obtuvieron con la versión anterior del núcleo y no se regeneraron.

6.5 Limitaciones de los controladores

Con el MPC no lineal activo la simulación en vivo corre a unas 1.4 veces el tiempo real en la máquina de desarrollo, de modo que las velocidades ×2 y ×4 no se alcanzan. Los controladores de aprendizaje dependen de los archivos de pesos en `models/`; si faltan, la interfaz los informa y vuelve al modo actitud. Con interferencia magnética y escalones de 1 m el PID de posición entra en un ciclo límite; es un hallazgo del banco, no un error de programación. PID, MPC y MPC no lineal se afinaron con una de las semillas del banco de comparación, y las políticas de RL tuvieron un solo entrenamiento cada una; las conclusiones del banco valen para estas sintonías y estas políticas.

6.6 Datos pendientes del dron real

- Protocolo y frecuencia de actualización del ESC.
- Constante de tiempo del motor medida en banco.
- Modo y tasa reales del barómetro BMP180 en el firmware.
- Sistema de posicionamiento disponible y su latencia, que decide si la compensación de aceleración puede usarse.

7 Despliegue y ejecución

7.1 Entorno de desarrollo

El proyecto es Python puro. Se recomienda un entorno virtual para aislar las dependencias:

```
git clone https://github.com/JesusJacome18/PAE-Quadcopter-SIM.git
cd PAE-Quadcopter-SIM
python -m venv .venv
.venv\Scripts\activate            # Windows
source .venv/bin/activate         # Linux / macOS
pip install -r requirements.txt
```

7.2 Dependencias

Tabla 12 | Dependencias declaradas en requirements.txt.

Biblioteca	Versión mínima	Uso
numpy	1.24	Núcleo de simulación
PySide6	6.5	Interfaz gráfica
pyqtgraph	0.13.3	Gráficas en tiempo real
PyOpenGL	3.1	Vista 3D (opcional)
matplotlib	3.7	Gráficas de barridos fuera de la interfaz
scipy	1.10	Identificación del modelo sustituto de RL
pytest	7	Pruebas

7.3 Ejecución desde Python

```
python main.py                       # tema oscuro
python main.py --light               # tema claro (proyector)
python main.py --vehicle luukkonen --play
python main.py --example 05          # abre el ejemplo 05 de examples/
python main.py --scenario archivo.json
python main.py --tour                # arranca el tour guiado
python main.py --no-welcome          # sin la bienvenida del primer arranque

# variables de entorno
PAE_NO_GL=1                          # fuerza la vista 3D sin OpenGL
QT_QPA_PLATFORM=offscreen            # sin pantalla (pruebas)
```

El núcleo se puede usar sin interfaz desde cualquier guion:

```
from quadsim.scenarios import make_tracking_sim
from quadsim.trajectories import make_trajectory
from quadsim.control.position import make_controller

traj = make_trajectory("lemniscate")
ctrl = make_controller("mpc", "pae_x450")
sim = make_tracking_sim(traj, ctrl, "wind", seed=1)
sim.set_trajectory(traj)
log = sim.run(traj.duration)   # dict de arreglos: t, pos_0..2, ref_0..2, ...
```

7.4 Pruebas y verificación

```
# Windows: set PYTHONPATH=.      Linux/macOS: export PYTHONPATH=.
python -m pytest -q                          # 223 pruebas
python experiments/model_verification.py     # verificación numérica (~1 min)
python experiments/benchmark_controllers.py --trajectories lemniscate circle \
    lemniscate_fast step_waypoints --scenarios nominal wind payload mag_emi \
    --seeds 1 2 3                             # banco completo (~25 min)
python experiments/sweep_cli.py --help       # barridos
python experiments/train_rl.py validate      # validación de las políticas
```

Las pruebas que miden tiempo de cómputo dependen del equipo y se omiten por defecto; se activan con la variable de entorno `PERF_TESTS=1`. Las pruebas de la interfaz corren sin pantalla con `QT_QPA_PLATFORM=offscreen`. Además de las pruebas automáticas, `docs/PROTOCOLO_PRUEBAS.md` reúne 87 casos de prueba manuales, agrupados por página y por función (arranque, menús, cada página, ejemplos, exportación, robustez). Cada caso indica qué tocar, con qué valor, qué debe aparecer y qué señal indica un defecto, y el protocolo termina con una tabla de resultados para marcar en el equipo donde se va a presentar el programa.

7.5 Organización de datos y modelos

Tabla 13 | Archivos de datos generados y consumidos.

Ruta	Contenido
models/rl_pure.npz, models/mpc_rl_residual*.npz	Pesos de las políticas de RL y metadatos del sustituto.
models/surrogate_id.json	Parámetros identificados del modelo sustituto.
experiments/results/benchmark_final/	Resultados del banco: runs.csv, summary.csv, summary.md y gráficas.
experiments/results/sweeps/<nombre>_<fecha>/	runs.jsonl (crudo, reanudable), results.csv, summary.md, best.json, gráficas.
docs/modelo/resultados_verificacion.json	Cifras de la verificación numérica del modelo.
Exportación CSV de la interfaz	Telemetría a 100 Hz de tiempo simulado, hasta 600 s.

7.6 Ejecutable para Windows (PyInstaller)

La distribución para usuarios sin Python es una compilación de PyInstaller en modo de una carpeta (*one-folder*), descrita en `packaging/QuadSIM-PRIMBIO.spec`. El resultado es la carpeta `dist/QuadSIM-PRIMBIO/`, con el ejecutable `QuadSIM-PRIMBIO.exe` y una subcarpeta `_internal/` que contiene el intérprete de Python, las bibliotecas y los datos que el programa necesita: la carpeta `examples/` con los once escenarios, la carpeta `models/` con los pesos de las políticas de aprendizaje por refuerzo, el PDF del modelo matemático y el icono `assets/icono.png`. El ejecutable lleva el icono `assets/icono.ico`. Se prefirió este modo al de archivo único porque arranca más rápido: no tiene que descomprimir las bibliotecas en una carpeta temporal en cada inicio. Una compilación de prueba en Linux ocupó unos 385 MB; se verificó que arranca y que ejecuta el ejemplo 06 sin pantalla.

La compilación se hace de dos formas equivalentes:

- **Integración continua.** El flujo `.github/workflows/build-windows.yml` corre en una máquina `windows-latest` de GitHub Actions con cada cambio en la rama principal: instala las dependencias, ejecuta un subconjunto de las pruebas automáticas, compila con PyInstaller, hace una prueba de humo (abre el ejecutable con el ejemplo 05 en marcha y comprueba que sigue vivo a los 20 s) y publica `QuadSIM-PRIMBIO-win64.zip` como artefacto. Cuando se publica una etiqueta de versión (`v*`), el mismo archivo se adjunta a una versión (*Release*) del repositorio.
- **Compilación local.** En un equipo Windows con Python 3.10 o posterior, el guion `build_exe.bat` de la raíz del repositorio crea un entorno virtual, instala las dependencias y PyInstaller, compila y comprime la carpeta en `QuadSIM-PRIMBIO-win64.zip`.

El punto de entrada es el mismo `main.py`, de modo que el ejecutable acepta las mismas opciones de línea de órdenes. En el programa compilado, los resultados de los barridos se guardan en `Documentos/QuadSIM-PRIMBIO/barridos` del usuario, porque la carpeta del ejecutable puede no tener permisos de escritura. Una compilación de PyInstaller solo corre en el sistema operativo en que se generó: por eso el ejecutable de Windows se construye en Windows.

Archivos de empaquetado presentes en el repositorio: `build_exe.bat`, `packaging/QuadSIM-PRIMBIO.spec`, `.github/workflows/build-windows.yml`.

7.7 Regeneración de estos documentos

```
PYTHONPATH=. python docs/registro/build_registro.py --todo
#   --capturas  capturas de la interfaz (make_capturas.py)
#   --figuras   diagramas y figura de portada (figuras.py)
#   --verificar corre pytest y guarda verificacion.json
```

7.8 Tecnologías utilizadas

Tabla 14 | Tecnologías.

Tecnología	Uso
Python 3	Lenguaje de todo el proyecto.
NumPy	Álgebra, integración, filtros, estimadores, MPC, red neuronal y PPO.
Qt 6 (PySide6)	Ventana, señales, hilos, temporizador, dibujo de instrumentos.
pyqtgraph	Gráficas de alto rendimiento.
OpenGL (PyOpenGL)	Escena 3D.
SciPy, Matplotlib	Identificación y gráficas fuera de la interfaz.
pytest	Pruebas automáticas.
LaTeX, pandoc	Documento del modelo y su versión Markdown.

8 Glosario de términos

Tabla 15 | Glosario.

Término	Definición
6-DOF	Seis grados de libertad: tres de traslación y tres de rotación.
AHRS	Sistema de referencia de actitud y rumbo: estimador de la orientación.
Anti-windup	Mecanismo que detiene el integrador de un PID cuando el actuador está saturado.
Biquad	Filtro digital de segundo orden con dos polos y dos ceros.
Cascada	Arquitectura de control con un lazo externo que da la referencia de uno interno.
Dryden	Modelo espectral de turbulencia atmosférica (MIL-F-8785C).
EKF	Filtro de Kalman extendido.
EMI	Interferencia electromagnética; aquí, sobre el magnetómetro.
ESC	Controlador electrónico de velocidad del motor sin escobillas.
FISTA	Algoritmo de gradiente proximal acelerado usado para el QP del MPC.
FOPDT	Modelo de primer orden con tiempo muerto.
FRD / NED	Marcos cuerpo (adelante-derecha-abajo) e inercial (norte-este-abajo).
iLQR	Regulador cuadrático lineal iterativo, método de optimización de trayectorias.
IMU	Unidad de medida inercial: giroscopio y acelerómetro.
Mezclador	Bloque que reparte empuje y momentos entre los motores.
MPC	Control predictivo basado en modelo.
Paso de pala	Frecuencia a la que pasan las palas de la hélice; origen del tono de vibración.
Pareto (frente)	Conjunto de configuraciones no dominadas en dos objetivos.
Planitud diferencial	Propiedad que permite obtener empuje y actitud a partir de la aceleración deseada.
PPO	Optimización de política proximal, algoritmo de aprendizaje por refuerzo.
RK4	Método de Runge–Kutta de cuarto orden.
RMSE	Raíz del error cuadrático medio.
SIMC	Reglas de sintonía de PID de Skogestad basadas en el modelo.
Sim2Real	Transferencia de un controlador entrenado en simulación al sistema real.
Factor de tiempo real (TR)	Segundos simulados por segundo de reloj.
ZOH	Retención de orden cero: el mando se mantiene constante entre muestras.